

■ Day 6 — Redshift Spectrum & Lakehouse Architecture

■ Topics Covered

- Redshift Spectrum
 - External Tables
 - Query S3 Directly
 - Glue Data Catalog
 - Partition Pruning
 - Parquet Optimization
 - Lakehouse Architecture
 - Medallion Architecture
 - Real Enterprise Pipelines
-

■ Real Industry Problem

Companies may store hundreds of TBs or PBs of data in Amazon S3.

Loading everything into Redshift becomes:

- expensive
- unnecessary
- difficult to maintain

Solution → Redshift Spectrum

■■ Real Lakehouse Architecture

Applications

↓

Kafka / CDC

↓

Amazon S3 Data Lake

↓

Parquet Files

↓

Glue Data Catalog

↓

Redshift Spectrum

↓

BI Dashboards

■ LAB 1 — Create S3 Data Lake Folder

Create folder:

s3://redshift-demo-bucket/spectrum/

■ customer_logs.csv

```
customer_id,customer_name,city,total_spent
101,Rohan,Mumbai,12000
102,Amit,Pune,18000
```

103, Neha, Delhi, 22000
104, Priya, Bangalore, 9000
105, Karan, Hyderabad, 15000

■ LAB 2 — IAM Role Setup

Required Policies:

- AmazonS3ReadOnlyAccess
- AWSGlueConsoleFullAccess

■ LAB 3 — Create External Schema

```
CREATE EXTERNAL SCHEMA spectrum_schema
FROM DATA CATALOG
DATABASE 'spectrum_db'
IAM_ROLE 'YOUR-IAM-ROLE-ARN'
CREATE EXTERNAL DATABASE IF NOT EXISTS;
```

■ LAB 4 — Create External Table

```
CREATE EXTERNAL TABLE spectrum_schema.customer_logs (
  customer_id BIGINT,
  customer_name VARCHAR(100),
  city VARCHAR(50),
  total_spent BIGINT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION 's3://redshift-demo-bucket/spectrum/';
```

■ LAB 5 — Query S3 Data Directly

```
SELECT *
FROM spectrum_schema.customer_logs;
```

■ Analytics Query

```
SELECT
  city,
  SUM(total_spent)
FROM spectrum_schema.customer_logs
GROUP BY city;
```

■ Partition Pruning

Without partitions Spectrum scans full S3 dataset.

Partitioning helps scan only matching folders.

Example:

s3://logs/year=2026/month=07/

```
CREATE EXTERNAL TABLE spectrum_schema.partitioned_logs (
  customer_id BIGINT,
  amount BIGINT
)
PARTITIONED BY (
```

```

        year INT,
        month INT
    )
    ROW FORMAT DELIMITED
    FIELDS TERMINATED BY ','
    STORED AS TEXTFILE
    LOCATION 's3://redshift-demo-bucket/logs/';

ALTER TABLE spectrum_schema.partitioned_logs
ADD PARTITION (year=2026, month=7)
LOCATION 's3://redshift-demo-bucket/logs/year=2026/month=07/';

SELECT *
FROM spectrum_schema.partitioned_logs
WHERE year=2026
AND month=7;

```

■ Why Parquet Preferred?

CSV Problems:

- large size
- slower scans
- row-based storage

Parquet Benefits:

- columnar format
- compressed
- predicate pushdown
- faster analytics

■ Medallion Architecture

- Bronze → Raw CSV/JSON
- Silver → Cleaned Parquet Data
- Gold → Business-ready Redshift tables

■ LAB 8 — Hybrid Query

```

SELECT
    c.customer_name,
    s.total_spent
FROM customers c
JOIN spectrum_schema.customer_logs s
ON c.customer_id=s.customer_id;

```

■ Real Production Challenges

- too many small files
- huge scans
- expensive queries
- schema evolution problems

Solutions:

- compact parquet files
- partition pruning
- Glue catalog management

■ Important Interview Questions

Q. What is Redshift Spectrum?

A. Query S3 data directly using Redshift SQL.

Q. Why Parquet preferred?

A. Columnar + compressed + optimized for analytics.

Q. What is Partition Pruning?

A. Scanning only matching partition folders.

■ Day 6 Outcome

- Learned Redshift Spectrum
- Created External Tables
- Queried S3 directly
- Learned Glue Catalog
- Learned Partition Pruning
- Learned Parquet Optimization
- Learned Lakehouse Architecture